

865. Smallest Subtree with all the Deepest Nodes

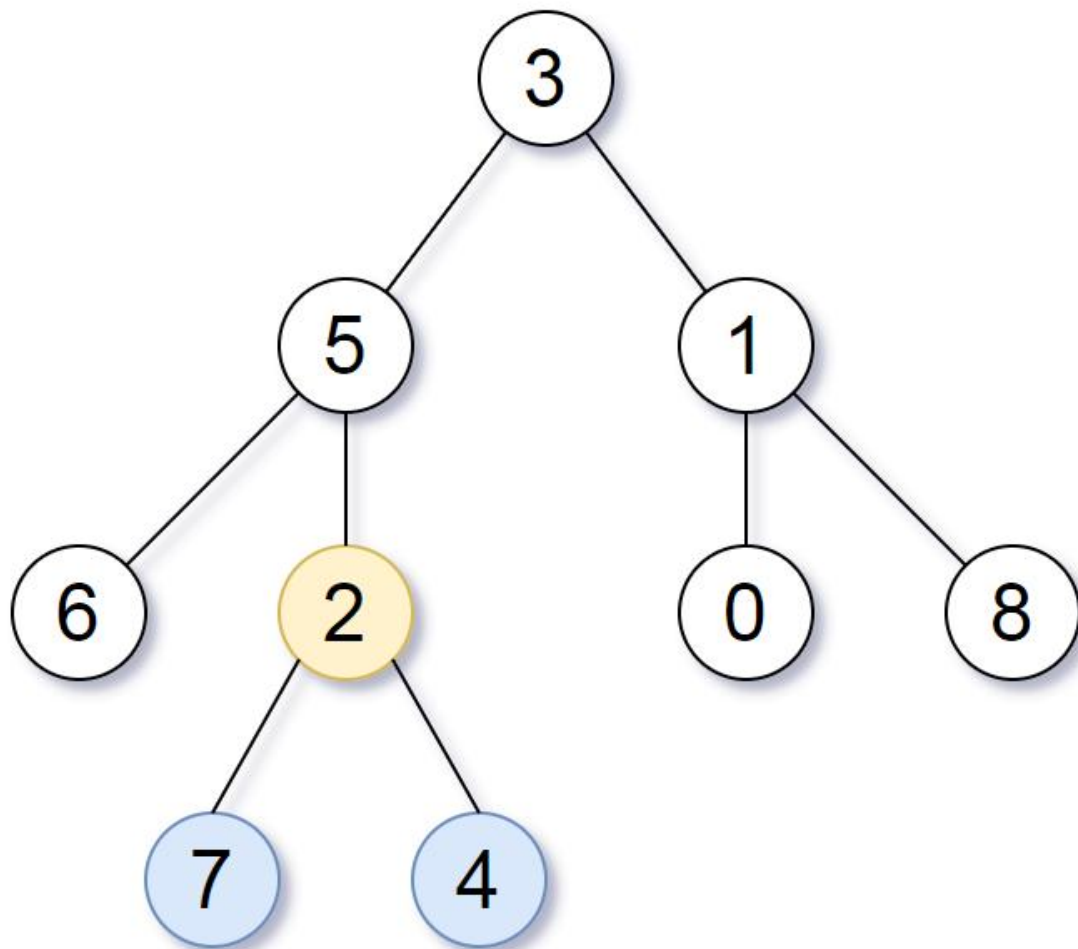
Given the root of a binary tree, the depth of each node is **the shortest distance to the root**.

Return *the smallest subtree* such that it contains **all the deepest nodes** in the original tree.

A node is called **the deepest** if it has the largest depth possible among any node in the entire tree.

The **subtree** of a node is a tree consisting of that node, plus the set of all descendants of that node.

Example 1:



Input: root = [3,5,1,6,2,0,8,null,null,7,4]

Output: [2,7,4]

Explanation: We return the node with value 2, colored in yellow in the diagram.

The nodes coloured in blue are the deepest nodes of the tree.

Notice that nodes 5, 3 and 2 contain the deepest nodes in the tree but node 2 is the smallest subtree among them, so we return it.

Example 2:

Input: root = [1]

Output: [1]

Explanation: The root is the deepest node in the tree.

Example 3:

Input: root = [0,1,3,null,2]

Output: [2]

Explanation: The deepest node in the tree is 2, the valid subtrees are the subtrees of nodes 2, 1 and 0 but the subtree of node 2 is the smallest.

Constraints:

- The number of nodes in the tree will be in the range [1, 500].
- $0 \leq \text{Node.val} \leq 500$
- The values of the nodes in the tree are **unique**.

Explanation in Hindi

हमें एक **Binary Tree** दी गई है।

Tree में जो node सबसे ज्यादा **depth (गहराई)** पर हैं, उन्हें **deepest nodes** कहते हैं।

👉 हमें ऐसा **सबसे छोटा subtree** लौटाना है जिसमें सारे **deepest nodes मौजूद हों**।

♦ आसान समझ

- पहले पता करो: **सबसे गहरे (deepest) nodes कौन से हैं**
- फिर उनका **Lowest Common Ancestor (LCA)** ढूँढो

- वही node answer होगा
-

♦ तरीका (Logic)

हम **DFS (Depth First Search)** का उपयोग करेंगे।

हर node से दो चीज़ें लौटाएँगे:

1. उस subtree की **depth**
 2. वह **node** जो deepest subtree का root है
-

♦ DFS में क्या करेंगे?

1. Left और Right subtree पर DFS लगाएँ
2. उनकी depth compare करें:
 - अगर **left depth ज्यादा** → left वाला answer
 - अगर **right depth ज्यादा** → right वाला answer
 - अगर **दोनों बराबर** → current node ही answer
3. Depth में +1 जोड़ते जाएँ

Java code

```
class Solution {  
    private static class Pair {  
        int depth;  
        TreeNode node;  
        Pair(int d, TreeNode n) {  
            depth = d;  
        }  
    }  
}
```

```
        node = n;
    }
}
```

```
private Pair dfs(TreeNode root) {
    if (root == null) return new Pair(0, null);
    Pair l = dfs(root.left);
    Pair r = dfs(root.right);
    if (l.depth > r.depth) return new Pair(l.depth + 1, l.node);
    if (r.depth > l.depth) return new Pair(r.depth + 1, r.node);
    return new Pair(l.depth + 1, root);
}
```

```
public TreeNode subtreeWithAllDeepest(TreeNode root) {
    return dfs(root).node;
}
}
```